

Non-mandatory exercises: B

Types and explicit type conversion

1.

```
greeting = "Hello!!!"  
print(type(greeting))
```

The code above will result in the following print-out:

```
<class 'str'>
```

What is printed in the code below?

```
year = 2026  
print(type(year))
```

```
year = str(year)  
print(type(year))
```

2. What is printed? (The `␣` just indicates a normal space.)

```
year = 2026  
  
print("Hello!!" + "␣" + "Welcome!!")  
  
print("It␣is␣now␣year␣" + str(year))  
  
print(1 + 2)  
  
print("1" + "2")  
  
print(year + 1)  
  
print(str(year) + "1")
```

Non-mandatory exercises: B

3. The code below does not work. It results in the error: `TypeError: can only concatenate str (not "int") to str`. Add an **Explicit Type Conversion** to make the code work.

```
birth_year = 1842
print("I was born " + birth_year) #Incorrect
```

4. Which of the following code lines result in an error? (There are two **TypeError**s and two **ValueError**s. Can you spot which lines result in which error?)

- ☐ "a" + "b"
- ☐ "a" + 1
- ☐ 1 + 2
- ☐ "1" + 2
- ☐ "1" + "2"
- ☐ str(1) + "2"
- ☐ 1 + int("2")
- ☐ int("a") + 1
- ☐ "a" + str(1)
- ☐ int("a") + int("b")

Non-mandatory exercises: B

5. You have a file called **age.py** with the following content:

```
current_year = 2026
message = "Write the year you were born, and press enter:"

if __name__ == "__main__":
    # Code not working
    print(message)
    year_born = input()  # always return string, input()
    age_this_year = current_year - year_born
    print("This year you are turning:" +
          age_this_year)  # convert to string
```

If you try to run it, from the Unix prompt with

```
python3 age.py
```

it will result in an error. Add two explicit type conversions to make the code work.

6. Write a similar program as above. But make two questions, one that asks for the users name, and one that asks for the users age. Then print out a greeting message that includes the users name and your name, and also print the age difference between you and the user (you can pretend to be someone else than yourself, for instance a fictional character.)

```
my_name = "Harry Potter"
my_age = 25

name = input("What is your name? ")
age = int(input("How old are you? "))

age_difference = my_age - age

print("Hello", name + "!")
print("My name is", my_name)

print("The age difference between us is", abs(age_difference), "years.")
```

Non-mandatory exercises: B

Functions

7. In the file `print_greetings.py`, two functions are **defined**:

```
print_formal_greeting()
and
print_informal_greeting().
```

These functions are then later **called**.

```
def print_formal_greeting():
    print("Good day to you!")

def print_informal_greeting():
    print("Hi!")

if __name__ == "__main__":
    print_informal_greeting()
    print_formal_greeting()
    print_informal_greeting()
```

When you run `python3 print_greetings.py` in the Unix prompt, Python considers the file as the main program, so `__name__ == "__main__"` becomes true and the three function calls inside the block execute, printing "Hi!", "Good day to you!", and "Hi!". However, when you import the file in `ipython3`, Python only loads the function definitions and `__name__` is not `"__main__"`, so the function calls are skipped and nothing is printed automatically, but the functions can still be called manually after importing.

What happens if you run

```
python3 print_greetings.py
```

in the Unix prompt? What happens if you import the file in `ipython3`?

8. Open a file in `micro` that is called `print_bye.py`. Define a function that prints a good bye message, similar to the functions above. Add a part after

```
if __name__ == "__main__":
```

where the function is called.

Call it from the Unix command line: `python3 print_bye.py`

Also import it in `iPython3` `import print_bye`.

And then call the function you defined from the python prompt.

Running the file → Python executes the main block automatically.
Importing the file → Python only loads the functions; you decide when to call them.

We use:

```
if __name__ == "__main__":
```

to control when a Python file should run its code. we use it Because it allows us to reuse code without automatically executing it.

Non-mandatory exercises: B

9. There are three errors in the code below. What is incorrect?
(Two leading to `SyntaxError` and one leading to `IndentationError`)

```
# incorrect function definition
def print_informal_greeting():
    print("Hi!")

if __name__ == "__main__":
    print_informal_greeting()
```

10. Let's say you have the file `print_greetings.py`, with the following content:

```
def print_informal_greeting():
    print("Hi!")

if __name__ == "__main__":
    print_informal_greeting
```

there is no parentheses (), but it would not give the error, it would print the name of the function, return that

What happens when it is run in the Unix prompt? (`python3 print_greetings.py`)

When you run `python3 print_greetings.py`, Python executes the code because `__name__ == "__main__"` is true. However, the line `print_informal_greeting` does not call the function because it is missing parentheses (). Without (), Python only refers to the function object instead of executing it, so the program prints nothing and no error occurs. If it was written as `print_informal_greeting()`, then the function would run and print "Hi!".

Non-mandatory exercises: B

Functions with parameters

11. The two functions that are defined below, both have one **parameter**. What **arguments** are passed to these functions when they then are called in the code. What is printed?

```
def print_formal_greeting(person):  
    print("Good day to you" + person + "!")  
  
def print_informal_greeting(person):  
    print("Hi" + person + "!")  
  
if __name__ == "__main__":  
  
    print_formal_greeting("Ada")  
  
    print("---")  
    person_to_greet = "Mary"  
    print_informal_greeting(person_to_greet)  
    print_formal_greeting(person_to_greet)
```

12. Update the code from the question above, where you wrote a goodbye message, so that your function has one parameter. Call the function at least twice, and pass different attributes each time.

Non-mandatory exercises: B

13. Which of the following will run without error and print "Charles". (In two cases, the result will instead be a `TypeError`.)

- ☒

```
def announce_name(name):  
    print(name)  
  
announce_name("Charles")
```
- ☐

```
def announce_name():  
    print("Charles")  
  
announce_name("Charles")
```

error
- ☒

```
def announce_name():  
    name = "Charles"  
    print(name)  
  
announce_name()
```
- ☐

```
def announce_name():  
    name = "Charles"  
    print(name)  
  
announce_name("Charles")
```

error, TypeError: announce_name() takes 0 arguments but 1 was given

14. What will be printed when the code below is run?

```
def print_welcome(name, city):  
    print("Hello_" + name +  
          "!_Welcome_to_" + city + "!")  
  
print_welcome("Mary", "Oslo")  
print_welcome("René", "Stockholm")
```

Non-mandatory exercises: B

15. Let's say you have defined the following function:

```
def print_temperature(city, temperature):  
    print("The temperature in: " + city + " is "  
          + str(temperature) + " degrees")
```

Which two of the following will result in an error. (One will not result in an error, but the message printed will be strange. Which one is that?)

- ☐ `print_temperature("Stockholm", 15)`
- ☒ `print_temperature("Berlin", "21")`
- ☐ `print_temperature("Warsaw")`
- ☒ `print_temperature(30, "Milano")`
- ☒ `print_temperature("35", "Malaga")`

1 is correct

2. No error, because: `str(temperature)` converts "21" to a string. Output: The temperature in Berlin is 21 degrees. But it is strange because temperature should normally be a number.

3. `TypeError: missing 1 required positional argument: 'temperature'`

4. `city = 30`, tries to add a string and an integer.

5. The temperature in 35 is Malaga degrees

Actually, because both are strings, this one does not give an error, but the message is wrong.

Scope of variables

16. The code below results in a `NameError`. Why?

```
def year_counter(current_year):  
    next_year = current_year + 1  
    print("Next year, it will be: " +  
          str(next_year))
```

```
year_counter(2026)
```

```
print(next_year) # Results in a NameError
```

The code gives a `NameError` because `next_year` is a local variable inside the function `year_counter()`., Python looks for `next_year`, but it does not exist outside the function.

Non-mandatory exercises: B

Functions with return values

17. What is printed below?

```
def get_following_year(current_year):  
    next_year = current_year + 1  
    return next_year  
  
following_year = get_following_year(2026)  
print(following_year)  
  
print("A future year: " +  
      str(get_following_year(5000)))
```

2027
A future year: 5001

18. Write a function with one parameter: the year a person was born.

Let the function return the age of this person (more specifically what age the person turns into the current year.)

Call this function and print out the value that the function returns.

19. (This one is a bit more tricky) Write a function with one parameter: the year a person was born.

Let the function return the age that the person turns into the **next** year.

Use the function `get_following_year(current_year)`

```
18. def calculate_age(year_born):  
    current_year = 2026  
    age = current_year - year_born  
    return age
```

```
age = calculate_age(2000)
```

```
print(age)
```

```
19. def get_following_year(current_year):  
    return current_year + 1
```

```
def calculate_age_next_year(year_born):  
    current_year = 2026  
    next_year =  
    get_following_year(current_year)  
    age = next_year - year_born  
    return age
```

```
age = calculate_age_next_year(2000)
```

```
print(age)
```

Non-mandatory exercises: B

20. Let's say you have the following function:

```
def get_previous_year(current_year):  
    previous = current_year - 1  
    return previous
```

Two of the following code examples will print out 2025? Which of them? (The other two will result in a NameError.)

- ☐ `previous_year = get_previous_year(2026)`
`print(previous_year)`
- ☐ `get_previous_year(2026)` `print(previous)` error, previous not defined
- ☐ `print(get_previous_year(2026))`
- ☐ `previous_year = get_previous_year(2026)`
`print(previous)` error, previous not defined